Deep Learning and its Applications to Signal and Image Processing and Analysis
361.2.1120
Final Project

Osher sidi –318420239

Daniel Bilik – 213196207

Github- https://github.com/danielbilikId/SRFlowGenerator-for-div2k-SR.git

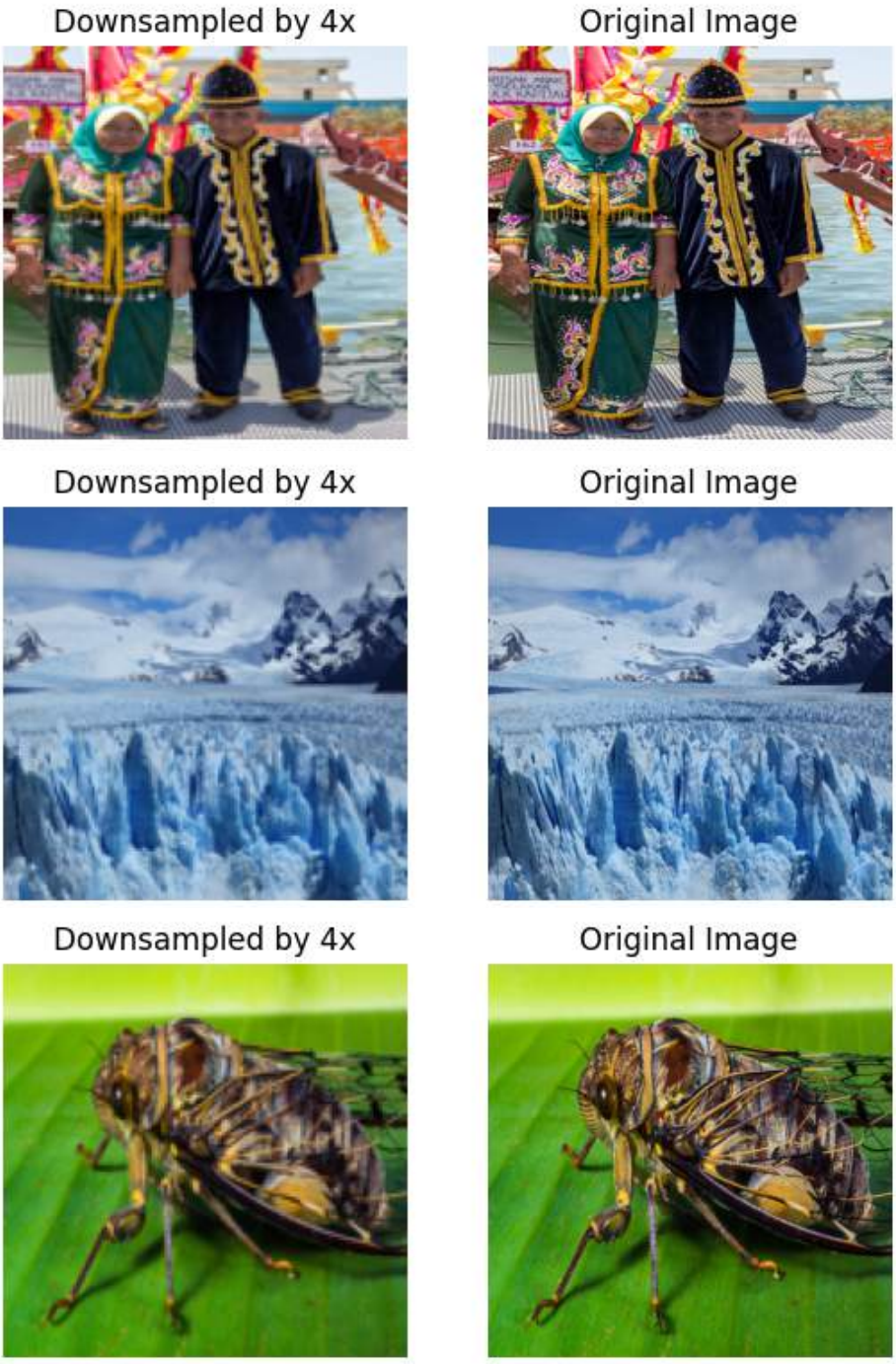
Abstract:

We chose the challenge of Super-Resolution Image Enhancement using the DIV2K dataset, where the goal is to restore fine details and textures to 4x downsampled low-resolution images, with performance assessed by PSNR, SSIM, and FID. For this, we utilized two model architectures: a vanilla – CNN based SRGAN inspired generator submitted to the Kaggle site. And we propose a model inspired by SRFlow that incorporates RealNVP normalizing flow layers, designed to learn complex, invertible mappings for enhanced detail and diverse texture generation.

Introduction, Objective, and Data Description:

1. The challenge addressed in this project is Super-Resolution Image Enhancement. Specifically, the objective is to enhance the resolution of low-quality images by performing a 4x upsampling operation. This involves taking a low-resolution input image and generating a high-resolution output that accurately restores fine details and texture lost during downsampling, thereby improving visual quality.

2. DIV2K (Diverse 2K Resolution Image Dataset). Comprises 900 high-resolution of width or height of at least 2000 pixels (HR) images, split into 800 for training and 100 for validation. These images cover a wide range of content, including natural scenes, urban landscapes, and human subjects, ensuring the trained models learn robust feature representations. For training, low-resolution (LR) counterparts are generated by downsampling the original high-resolution images by a factor of 4 using bicubic interpolation. This paired data allows the models to learn the mapping from low-resolution to high-resolution space.



Visualization of a few images from the dataset from Div2k_train. For preprocessing due to limited computational resources the (HR) images are resized to 1024x1024. With LR images downscaled to dimension 256x256.

3, Evalutation metrics:

Peak Signal-to-Noise Ratio (PSNR): A common metric that measures the ratio between the maximum possible power of a signal and the power of corrupting noise that affects the fidelity of its representation. Higher PSNR values indicate better reconstruction quality, though it often correlates poorly with human perception.

$$PSNR = 20\log_{10}\left(\frac{MAX_f}{\sqrt{MSE}}\right)$$

$$MSE = \frac{1}{mn}\sum_{0}^{m-1}\sum_{0}^{n-1}\left|\left|f(i,j) - g(i,j)\right|\right|^2$$

Structural Similarity Index Measure (SSIM): This metric assesses the perceptual similarity between two images, considering luminance, contrast, and structural information. SSIM values range from -1 to 1, with 1 indicating perfect similarity, providing a more perceptually relevant measure than PSNR. The closer SSIM is to 1, the better. The SSIM is defined by:

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

Where:

- μ_x and μ_y are the average intensities of x and y
- σ_x^2 and σ_y^2 are the variances of xx and yy
- σ_{xy} is the covariance of x and y
- $C_1 = (K_1L)^2$ and $C_2 = (K_2L)^2$are two small constants to stabilize the division (typically K_1=0.01, K_2=0.03, and L is the dynamic range of the pixel values)

Fréchet Inception Distance (FID): FID measures the "distance" between the feature distributions of real and generated images. Rather than comparing individual images, mean and covariance statistics of many images generated by the model are compared with the same statistics generated from images in the ground truth or reference set. Lower FID scores indicate higher quality and more diverse generated images, suggesting that the generated image distribution is closer to the real image distribution. It assumes that the features extracted from an Inception network follow a multivariate Gaussian distribution.

$$\text{FID} = |\mu_r - \mu_g|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r\Sigma_g\right)^{\frac{1}{2}}\right)$$

Where μ_r and μ_g are the means of the real and generated image features respecitvly. And Σ_r and Σ_g are the variances of the real and generated images respectively.

Model 1 – Vanilla SRModel:

1. **Provide a clear explanation of your model architecture or the modifications made to an existing one**

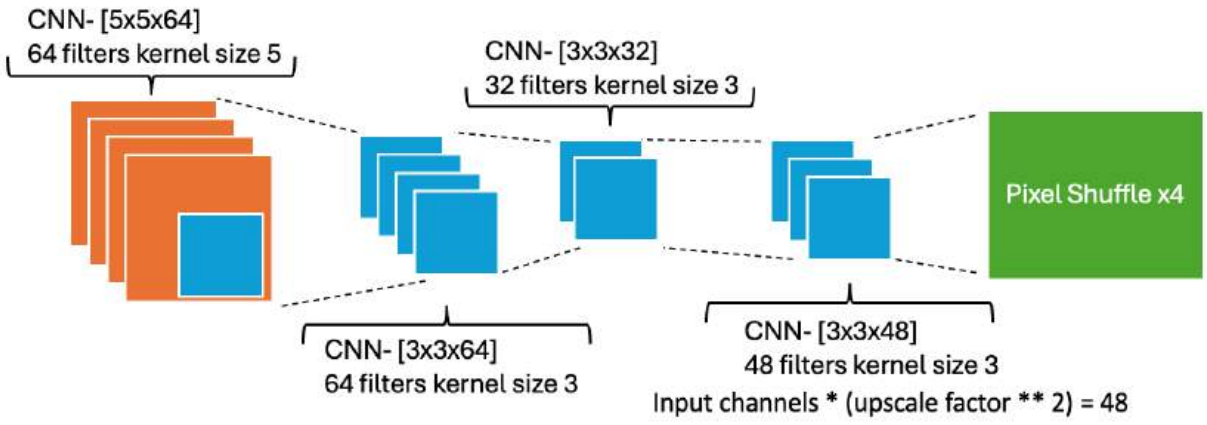
The SRModel is a vanilla implimentation of the generator module of the SRGAN [3] submitted to the div2k kaggle competition page. The submitted implimentation: Link to kaggle page

The SRModel is a lightweight convolutional neural network (CNN). It is composed of four main convolutional layers followed by a pixel rearrangement module (DepthToSpace) to perform the upsampling. 4 Convolutional Layers (conv1–conv4) with ReLU in between them.
These extract and process features from the input image:

1. conv1: First layer, 3 → 64 channels, with 5x5 kernel.
2. conv2: 64 → 64 channels, with 3×3 kernels.
3. conv3: 64 → 32 channels, with 3×3 kernels.
4. conv4: 32 → 48 with 3×3 kernels.

Followed by a DepthToSpace Layer (Pixel Shuffle)
This module reshapes the output of conv4, Converts high-channel, low-resolution data into a lower-channel, high-resolution image. If upscale_factor=4, it transforms from shape (N, C×4^2, H, W) to (N, C, 4H, 4W). where N is number of input images, C is 3 (RGB images), H, W are 512 as from original images.

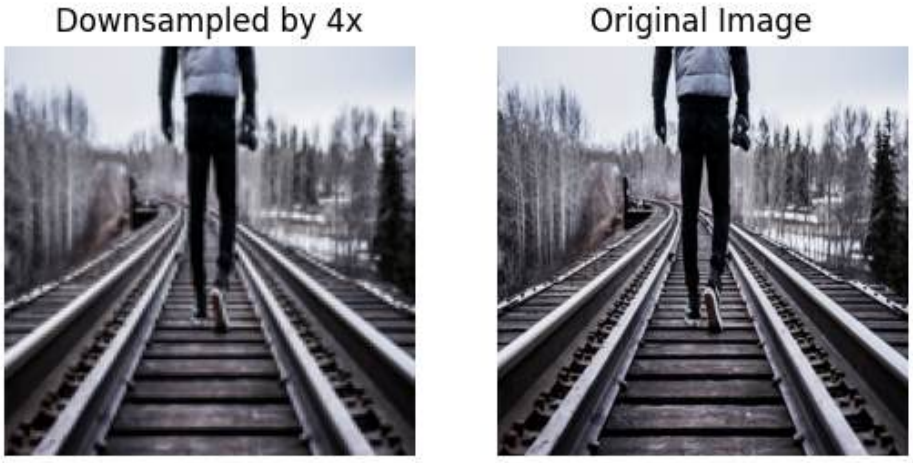
2. **Include a simplified diagram illustrating the model structure.**



3. **Describe the data preprocessing steps, including visualizations if applicable.**

We split the Dataset of 900 HR images to 720 train, 90 val, and 90 for test.

The preprocessing function prepares paired low-resolution (LR) and high-resolution (HR) We first create an HR version by resizing the images to size 1024x1024. To generate the LR image, the original image is resized to smaller dimensions—based on a downscaling ratio (x4) —using bicubic interpolation to dimension 256x256. The result is a tuple of LR and HR image tensor pairs, both normalized to the [0, 1] range, suitable for model input and target supervision.



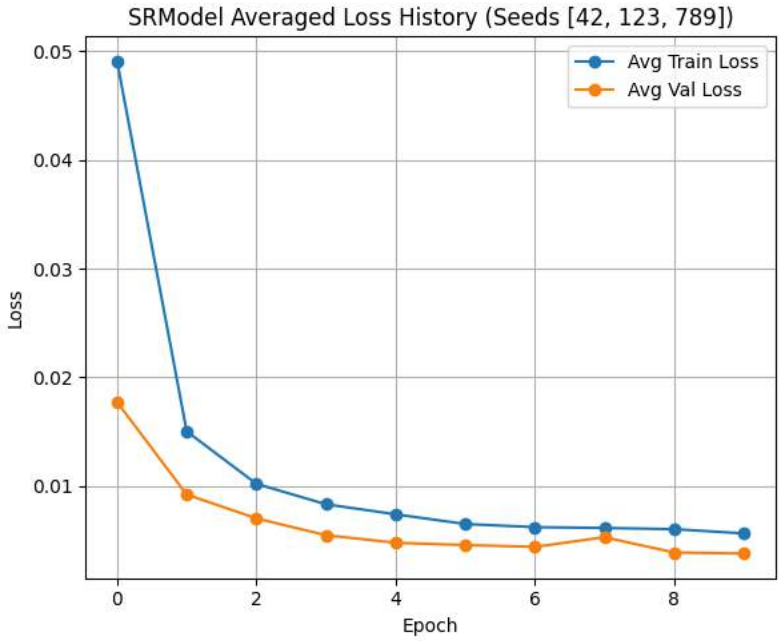
4. **Specify the hyperparameters used and the loss function applied.**

We train the super-resolution models in supervised learning on the DIV2K dataset using paired low-resolution (LR) and high-resolution (HR) images. Training is conducted for 10 epochs with a batch size of 16, using the Adam optimizer and a learning rate of 1e-3. The model's learning objective is the Mean Squared Error (MSE) loss, to focus on pixel-wise reconstruction. Defined by:

$$\mathcal{L}_{\text{mse}} = \frac{1}{N}\sum_{i=1}^{N}\left(y_i^{\text{pred}} - y_i^{\text{true}}\right)^2$$

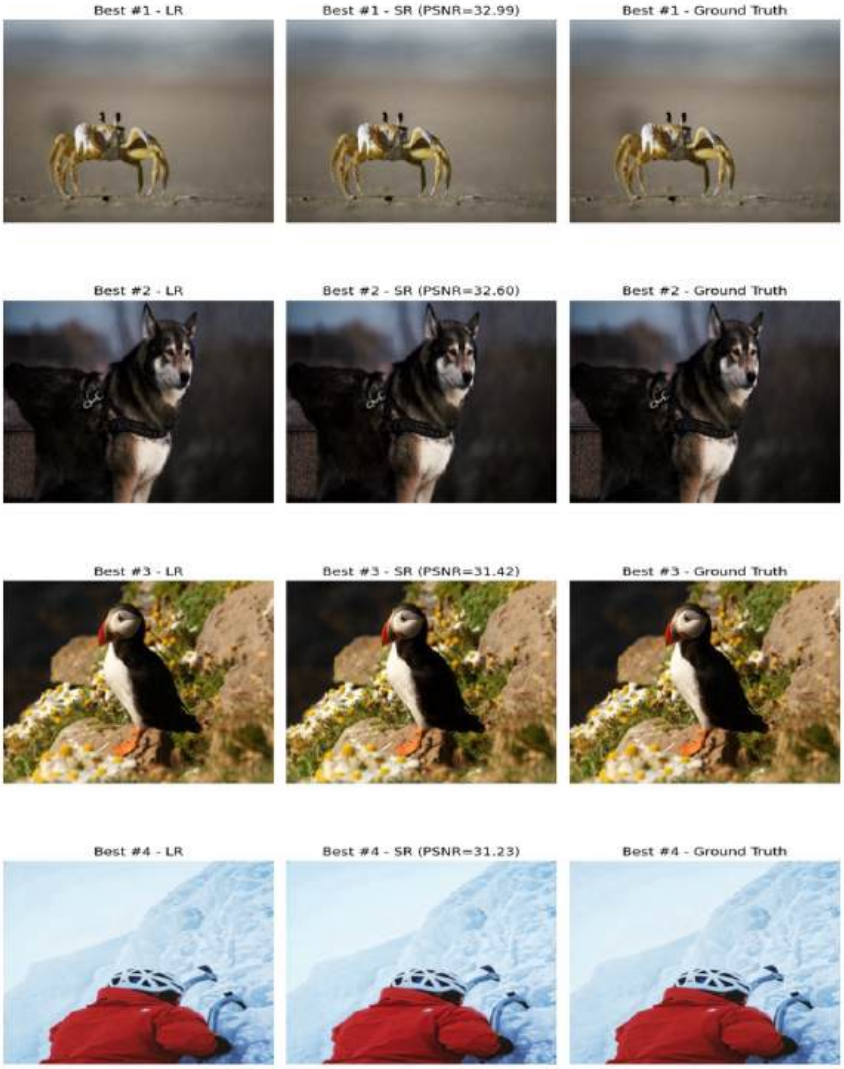
Throughout training, both training and validation losses are tracked per epoch, including PSNR and SSIM on the validation images, and model checkpoints and performance plots are saved for analysis. The framework also supports loading pre-trained weights to resume or fine-tune training.
We train both models on 3 random seeds: [42, 123, 789] and present the averaged reslts.

5. **Include a plot of the training loss over epochs to demonstrate convergence behavior.**

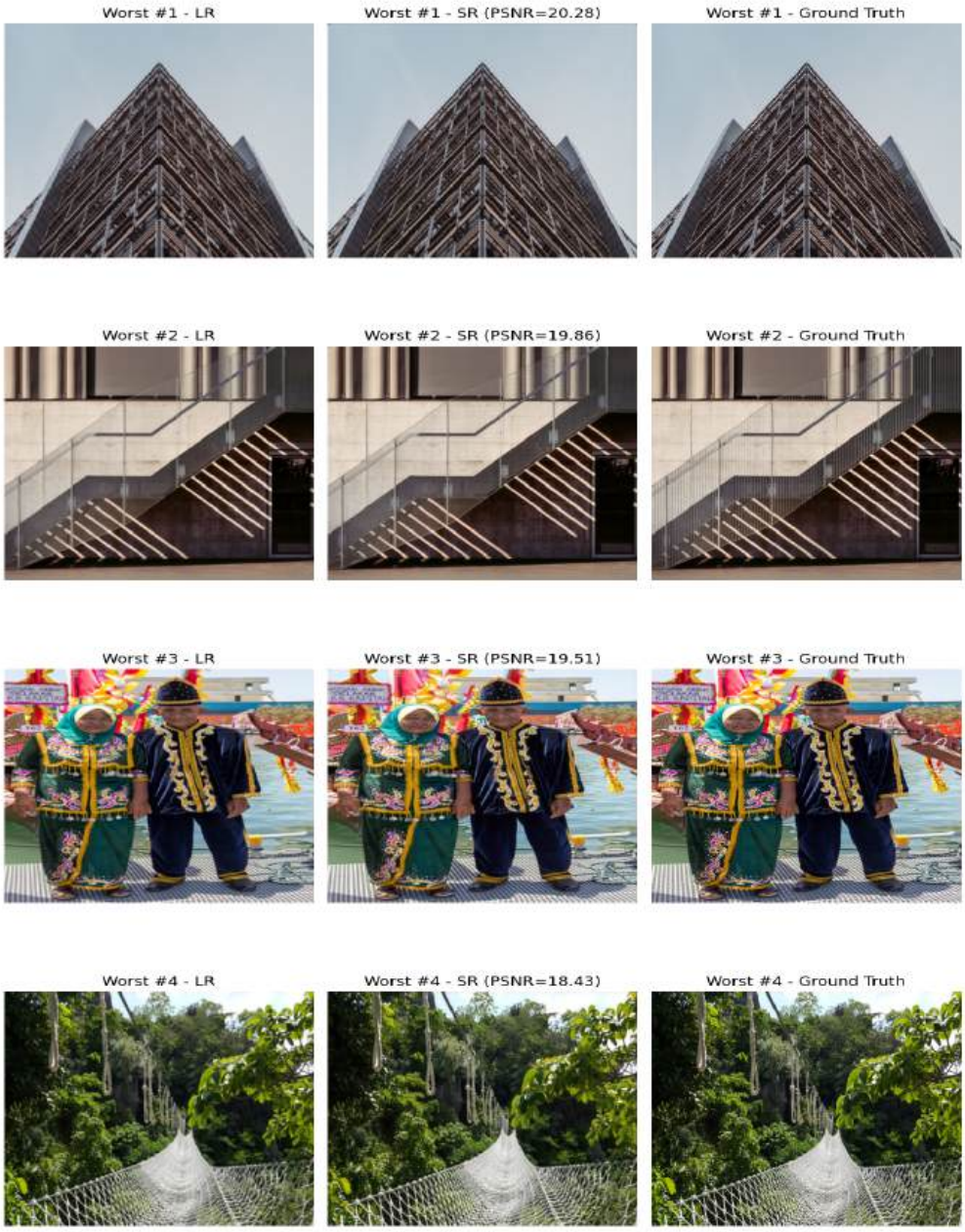


6. **Present test results, including both quantitative metrics and qualitative examples of both positive and negative performance.**

Model	PSNR	SSIM	FID
SRModel	24.1997 Std = 0.0372	0.7156 Std = 0.0031	39.8785 Std = 1.5279



we see that for images with highly focused centric object and smeared background the metr present high scores, because superresolutioning a smeared background is easy for the netwrok (its already not in focus)



In these images we get low PSNR and SSIM metrics likely because the images are rich in fine details whom are present in the entirety of the image, resulting in overall low success scores, as the vanial model cant recover with hig fidelity these small fine details.

Model 2 - SRFlow Generator

1. Provide a clear explanation of your model architecture or the modifications made to an existing one.

We propose the use of the SRFlowGenerator a module inspired by the generator function in the SRFlow paper [5]. Link to SRFlow github

While not entirely integrating normalizng flows into the SR task (due to very limited computational resources) the SRFlowGenerator model modifies the basic feed-forward convolutional structure of the vanilla SRModel by introducing residual learning and a deeper, hierarchical feature extraction, common in modern SRGAN generators like SRResNet or those found in ESRGAN.

Unlike SRModel which uses a sequential series of convolutions and ReLU activations directly leading to the upsampling, the SRFlowGenerator first employs an initial convolutional layer to extract base features.

The main modification is the sequence of four ResidualBlock units. Each ResidualBlock itself is a miniature network (two convolutions and a ReLU, with a skip connection) that allows the model to learn fine details and residuals more effectively. After these blocks, a final conv layer processes the aggregated residual features, and its output is then added back to the initial features from the first conv layer, forming a powerful long skip connection. This long skip connection helps to preserve low-frequency information by providing an alternate path for gradients.

Finally, the combined features are upsampled using a convolutional layer and PixelShuffle (similar to DepthToSpace in SRModel) to produce the high-resolution output.A single convolution layer (conv_first) maps the RGB input (3 channels) into a richer feature space with nfchannels (e.g., 32 filters).

Residual Learning (Residual Blocks)

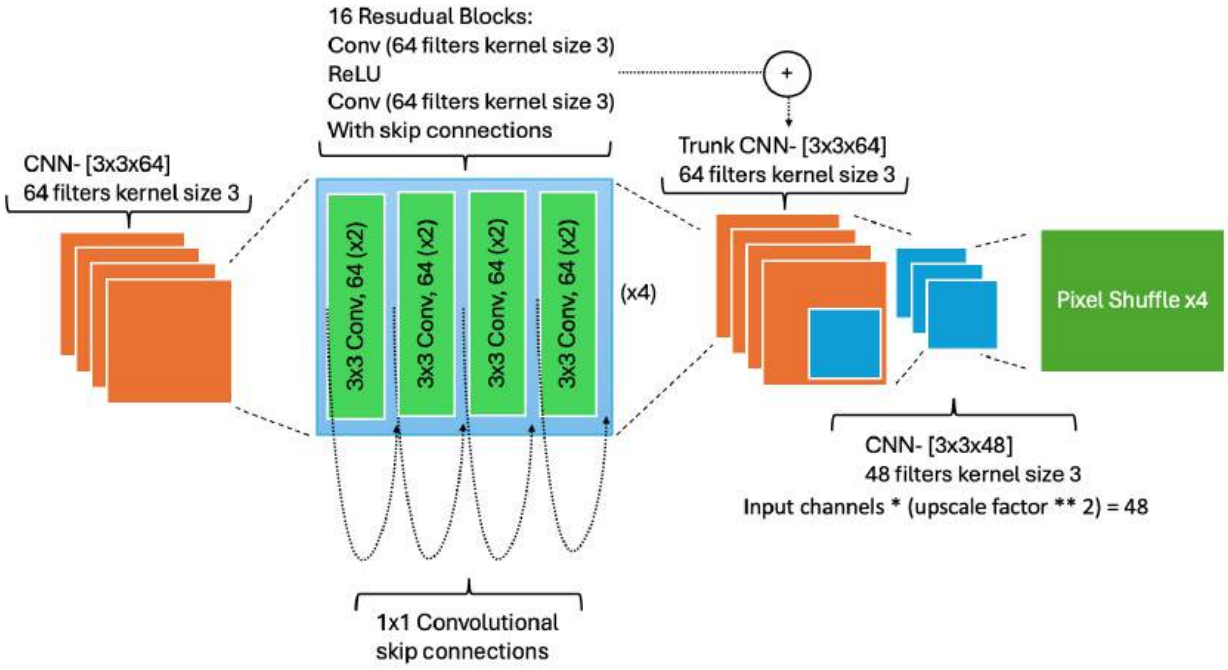
- The model uses **16 ResidualBlocks**, each made of:
    - Two 3×3 convolutions with ReLU.
    - A skip connection around the block.
- These are more efficient than heavier residual dense blocks (RDBs) and provide a deeper receptive field while preserving gradient flow.

- After the residual trunk, a trunk_conv layer aggregates features

- The output of the residual stack (trunk) is added to the initial features making a skip connection

Finally, the features are upscaled using a 3×3 convolution followed by a PixelShuffle, which rearranges the feature maps to the higher spatial resolution

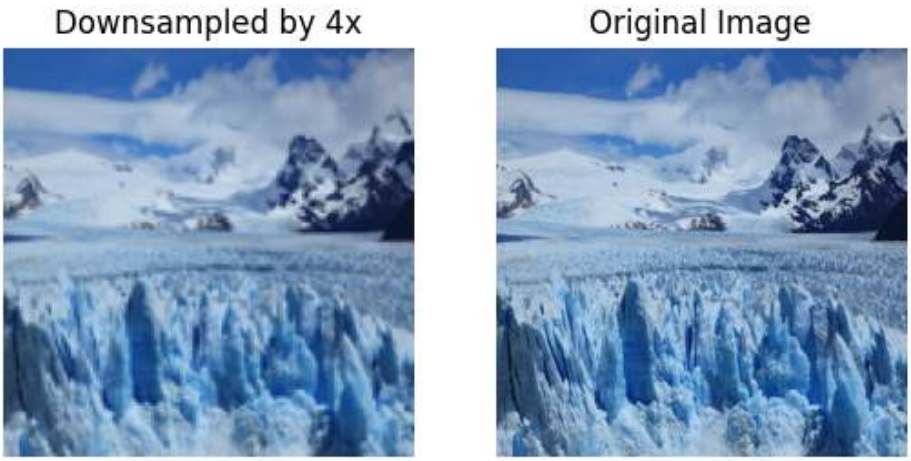
2. Include a simplified diagram illustrating the model structure.



*Note: there are 16 RDB's (Residual Blocks). Due to insufficient space we draw 4 in the diagram. Hence the (x4) near the RDB block.

3. **Describe the data preprocessing steps, including visualizations if applicable.**

We split the Dataset of 900 HR images to 720 train, 90 val, and 90 for test same as for the vanilla model. Creating the downsampled LR train images is also same as in the vanilla model.



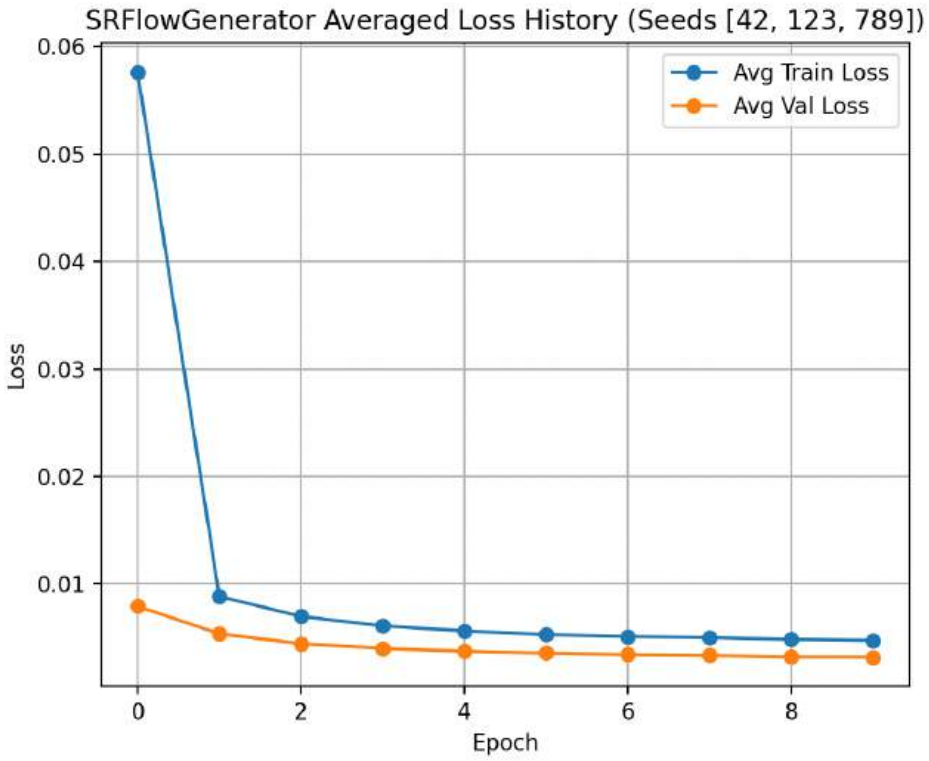
4. **Specify the hyperparameters used and the loss function applied.**

The training setup is the same as for the vanilla SRModel. We this model as well on 3 random seeds: [42, 123, 789] and present the averaged reslts.

Training is conducted for 10 epochs with a batch size of 16, using the Adam optimizer and a learning rate of 1e-3. The model's learning objective is the Mean Squared Error (MSE) loss, to focus on pixel-wise reconstruction. Defined by:

$$\mathcal{L}_{\text{mse}} = \frac{1}{N}\sum_{i=1}^{N}\left(y_i^{\text{pred}} - y_i^{\text{true}}\right)^2$$

5. Include a plot of the training loss over epochs to demonstrate convergence behavior.

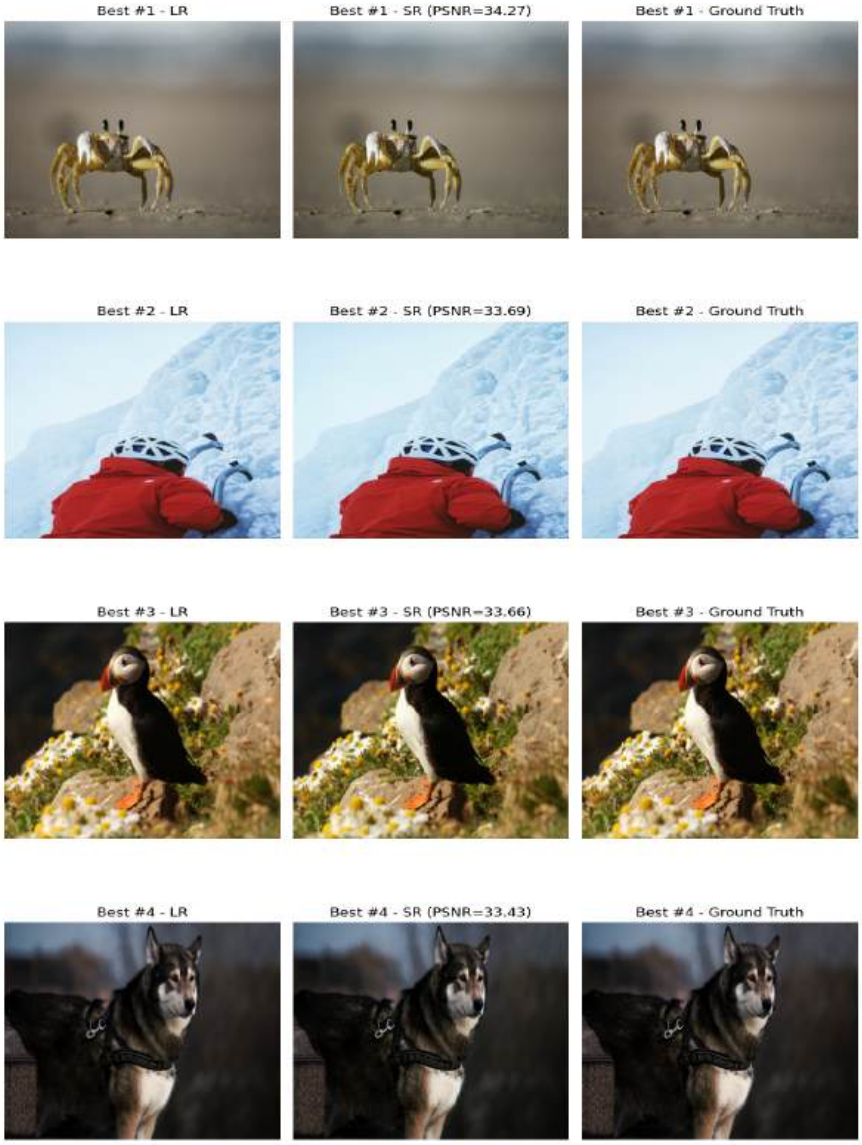


6. Present test results, including both the quantitative metrics and qualitative examples of both positive and negative performance.

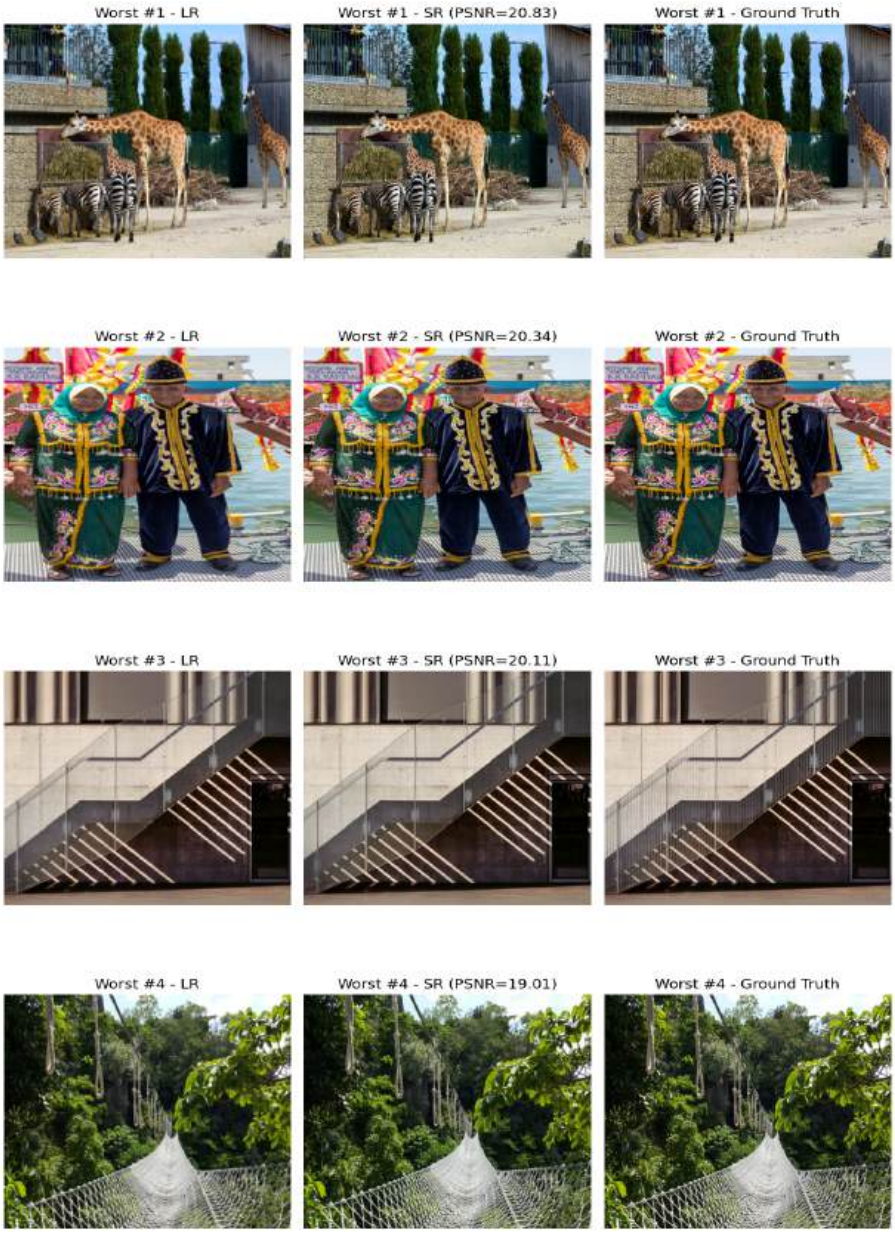
Average metrics across all 3 seeds [42, 123,789]:

Model	PSNR	SSIM	FID
SRFlowGenerator	24.9777 Std = 0.0434	0.7430 Std = 0.0056	26.8959 Std = 1.0489

We see that the average metrics of our proposed model are better then the vanilla model.



We see that both models preformed best on roguhly the same images. We see that these images include objects that are centered and highly focused on a blurry background. Blurry backgorund images result in SR ideally due to the background already being lowest possible resolution. Upscaliing and superresoliutioning a blurred backgroud will result in a blurred background also in HR. this blurred backgorund likely takes up most of the image, resulting in very high preformance metric scores, in PSNR, SSIM for entireyl preserving the image features as well as FID.

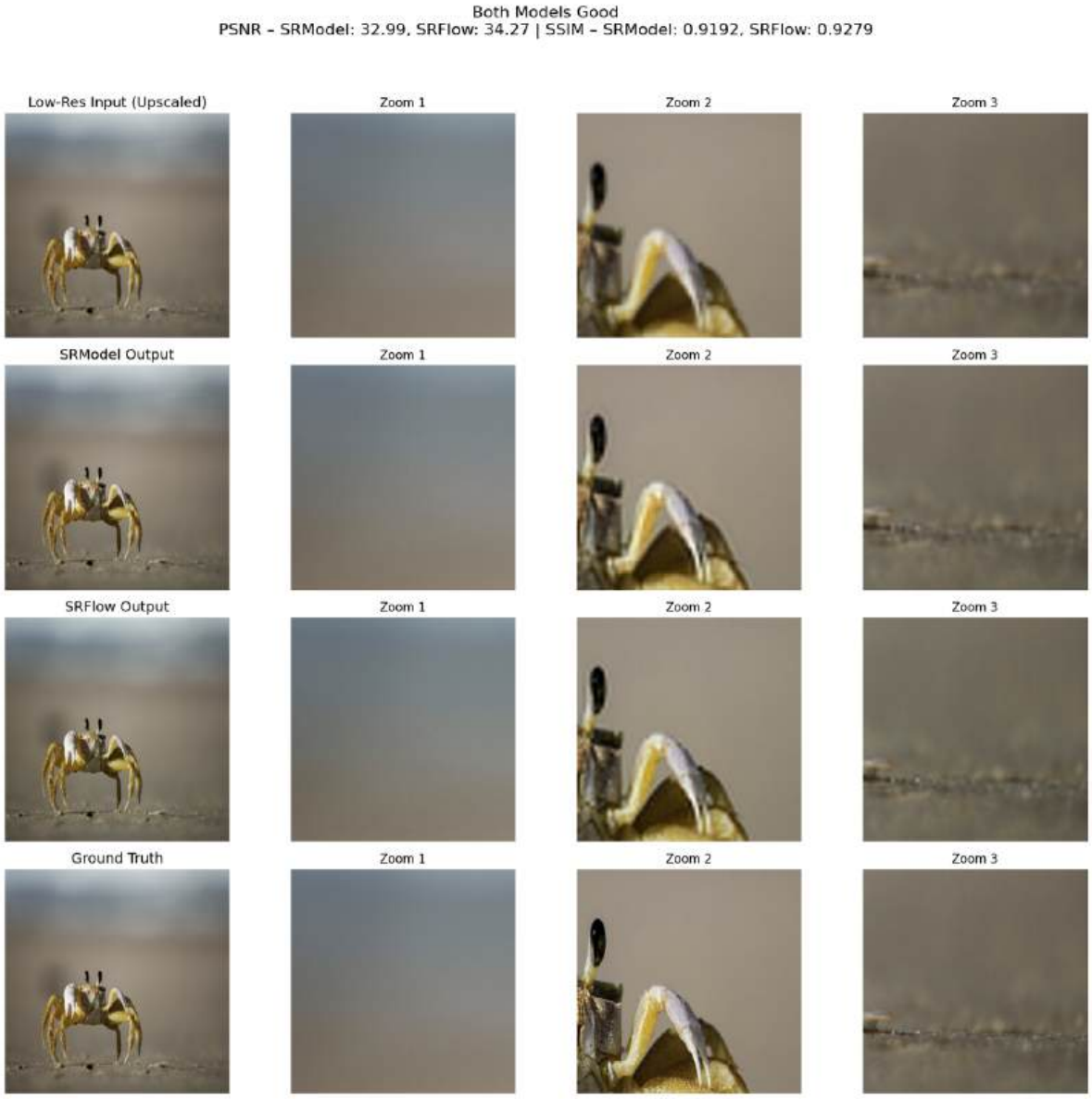


The worst preforming metrics vary in the images for the SRFlowGenerator model. We see that our implementation still struggles with very detailed images that have high resolution-high fidelity features scattered across the entirety of the image. The resulting super resolution images cant fully capture the full fidelity of these small details which likely results in the low PSNR scores. Conversely notice that the lowest PSNR score on the test set of images is still higher than the lowest score in the SRModel.

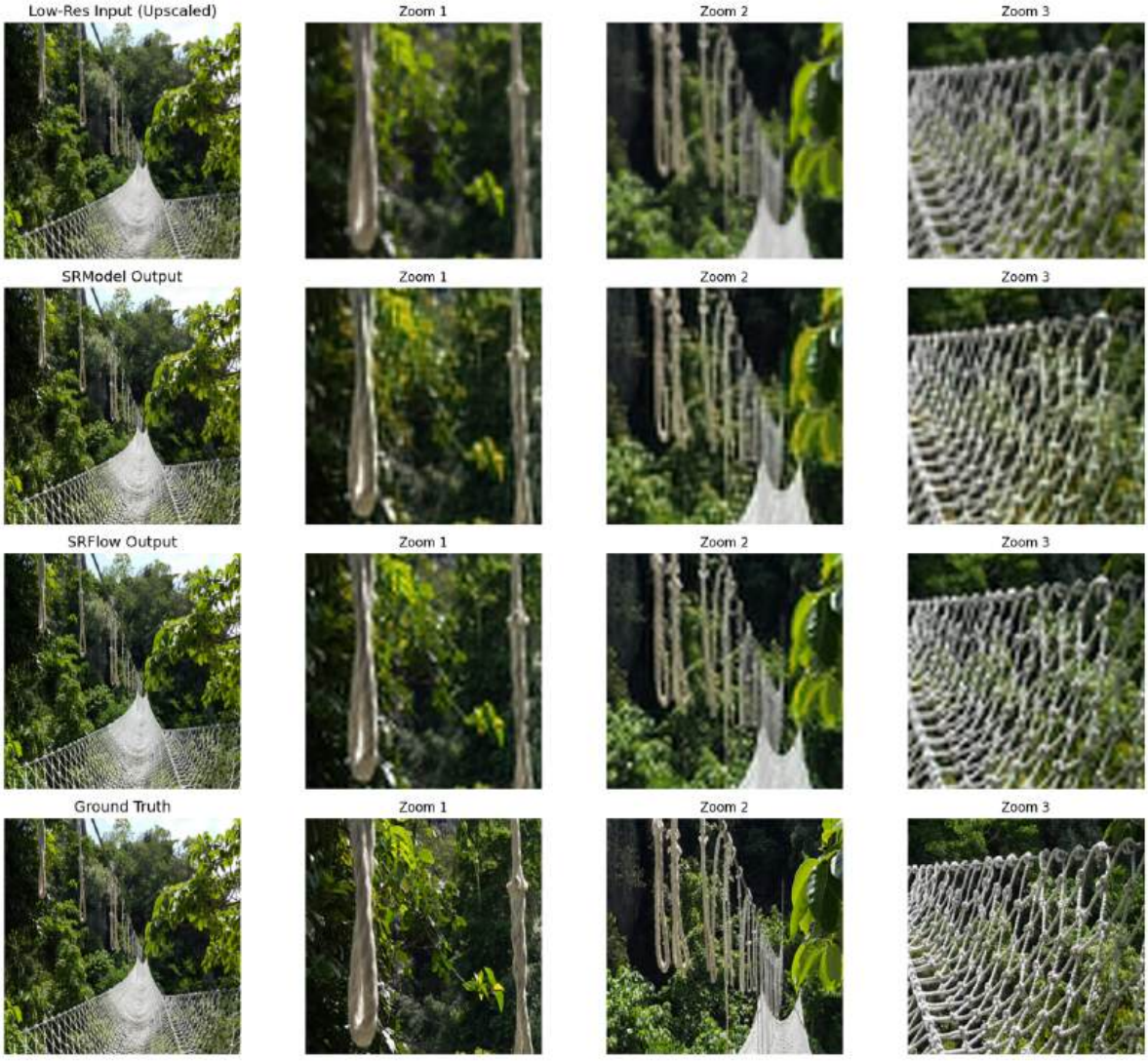
Results

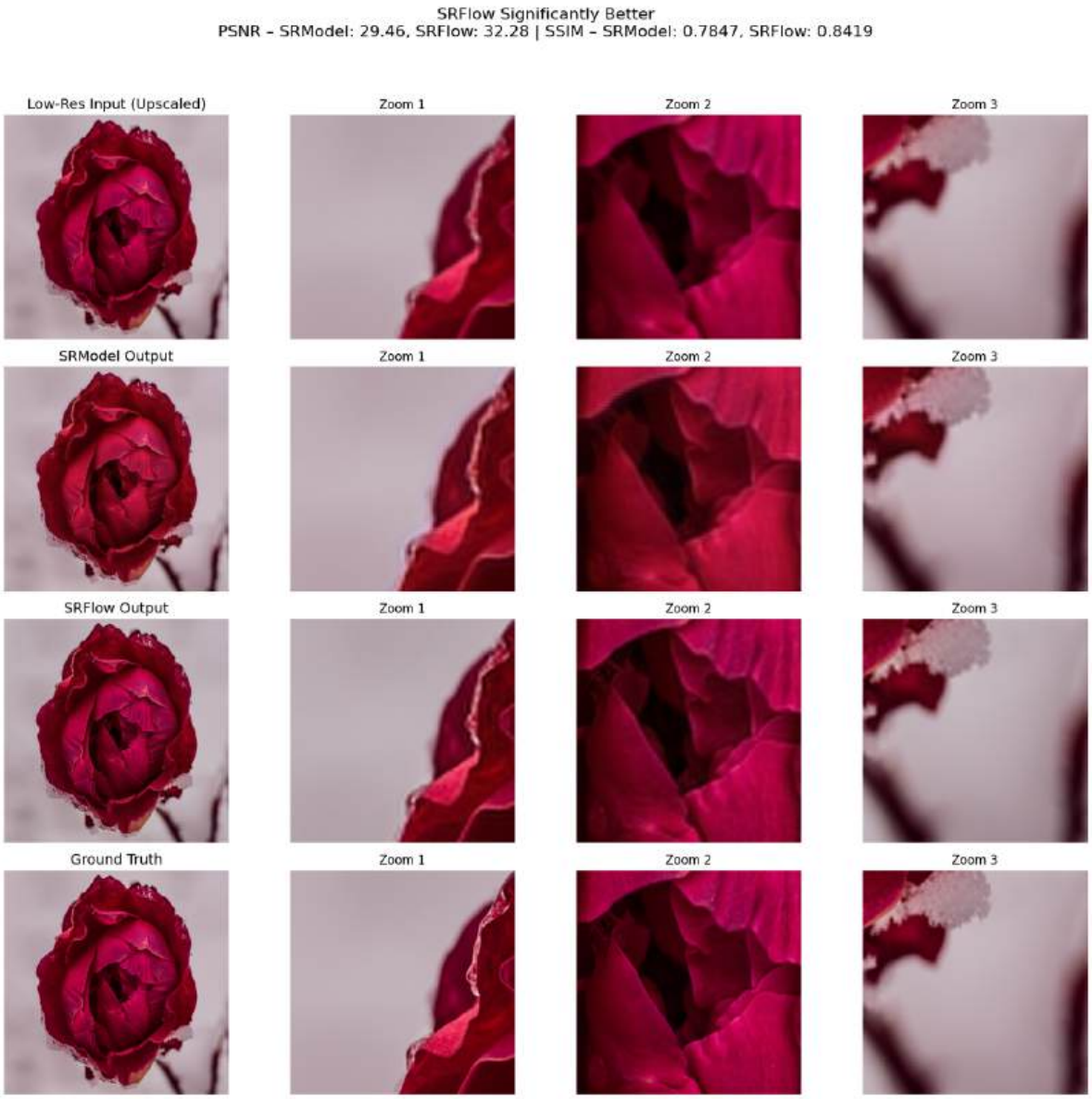
1. Show four examples of images:

   • One where both models performed well.
   • One where both models performed poorly.
   • One where Model 1 performed well and Model 2 did not (if none, please state). • One where Model 2 performed well and Model 1 did not (if none, please state).



Both Models Poor
PSNR – SRModel: 18.43, SRFlow: 19.01 | SSIM – SRModel: 0.4490, SRFlow: 0.4975





Comparing evaluations of the models on seed setting 42, with 10 training epochs, batchsize of 16 and LR of 1e-3. All 90 test images result in higher PSNR and SSIM scores for the SRFlowGenerator model (our proposed model) resulting in no test set iamges where the vanilla model outpreformed our model.

We classify the images in “preformed well”, and “preformed poor” based on the PSNR.

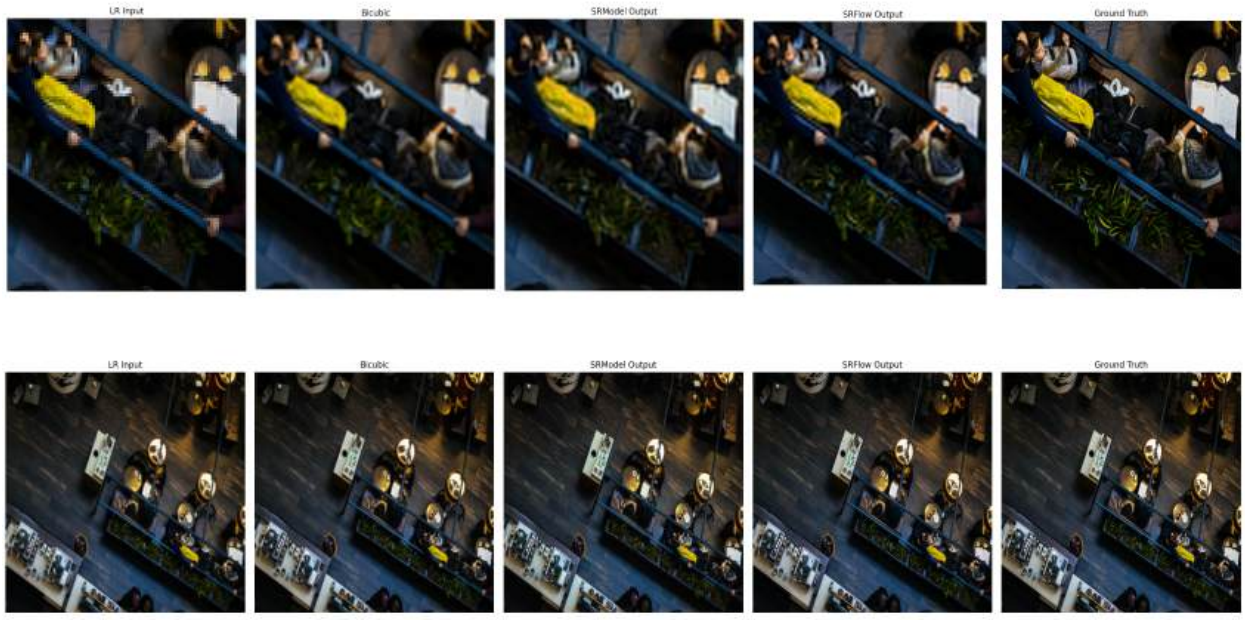
If the PSNR>30 the model preformed SR well & if the PSNR<25 the model preformed SR poor. If the discrepency in PSNR between vanilla SRModel and our proposed SRFlowGenerator > 2 dB then we classify that the model preformed “Significantly better”.

2. Compare the performance of both models (both quantitative and qualitative). Clearly state whether your variation outperformed the baseline model, and explain the reasons behind the outcome.

The results are an average on 3 training seeds. [42,123,789], reported allong with standard deviation.

Model	AVG. PSNR	AVG. SSIM	AVG. FID
SRModel (vanilla)	24.1997 Std = 0.0372	0.7156 Std = 0.0031	39.8785 Std = 1.5279
SRFlowGenerator (our proposed)	24.9777 Std =0.0434	0.7430 Std = 0.0056	26.8959 Std = 1.0489

Looking at generated images side by side:



We see that the SRFlowGenerator generates much higher detailed SR images than the vanilla SRModel. We see a greater focus on colors, shading, depth and detail generation that are missing from the SRModel. This is due to the increased depth of the model as well as the skip connection layers which allow for better generalizaiton when generating the SR image. The residual blocks we added improve texture representation and spatial coherence.

The main metric in which we see the imrpoovment is the FID score. The purpose of the FID score is to measure the diversity of images created by a generative model with images in a reference dataset. The lower the FID, the closer the SR is to the real image.

In this case we compare to the ground truth images. FID, being sensitive to texture statistics, benefits greatly from models that generate plausible fine details — even if those are hallucinated. SRFlow appears to generate crisper edges, sharper contours, and less smoothed-over textures compared to SRModel, all of which help explain the drop in FID compared to our implimentation. This is especially evident in Textured regions (like grass or hair) and Sharp geometric edges (windows or stripes). All of these boost the feature-level distribution alignment with the real images, which is exactly what FID measures. Thus our model outpreformed the baseline significantly.

Ablation Studies

1. Describe the ablation setup, specifying the component you varied and the motivation behind it. Include visualizations if applicable (e.g., for different augmentation methods).

In this ablation study, we attempt to add **the Mean Gradient Error (MGE)** loss component in the training objective of the `SRFlowGenerator` super-resolution model. The MGE penalizes differences in image gradients and tries to increase the sharpness of the SR images. The original setup used only **Pixel-wise loss** via MSE.

While MSE loss optimizes for PSNR, it often fails to preserve **perceptual sharpness** and **edge structures**. We can see that our SR images came out a little blurry in the previous experiment. We add MGE in order to try and improove the sharpness for fine details like: Thin lines Character strokes and High-frequency textures.

To introduce this we add a MGE loss term and weigh the loss function:

$$Loss = w_1 * MSE + w_2 * MGE$$

With weigths: $w_1 = 1, w_2 = 0.2$

Where

$$MGE(Y_{pred}, Y_{true}) = \frac{1}{2} \cdot E\left[\left|G_x(Y_{true}) - G_x(Y_{pred})\right| + \left|G_y(Y_{true}) - G_y(Y_{pred})\right|\right]$$

$$G_x = I * S_x$$

$$G_y = I * S_y$$

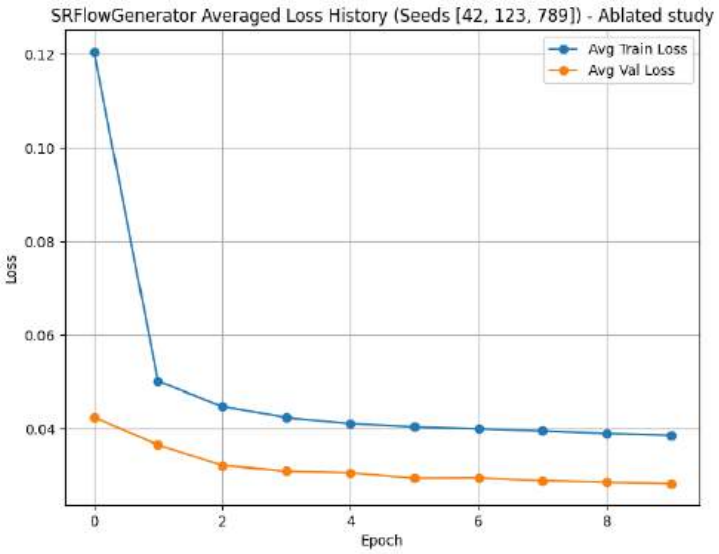
$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

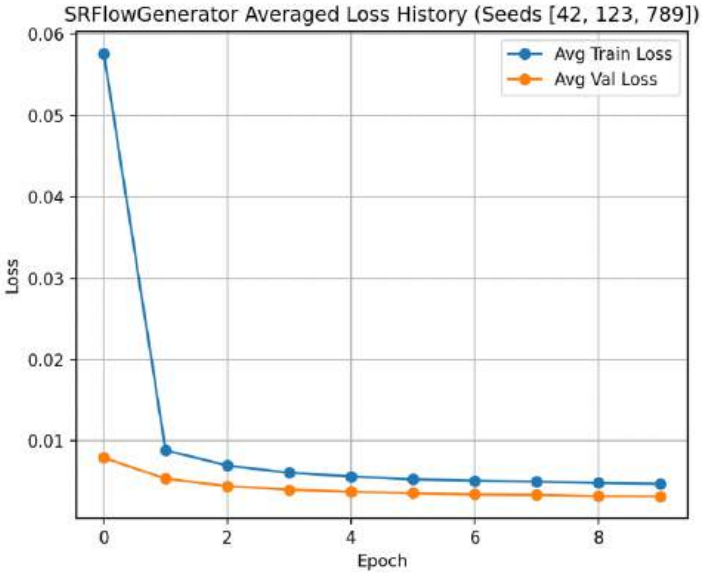
$$S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

where $*$ denotes the 2D convolution operation (with padding of 1 to maintain dimensions, and applied independently per channel, and MSE is defined as previously.

$$\mathcal{L}_{\text{mse}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_i^{\text{pred}} - y_i^{\text{true}}\right)^2$$

2. Plot and compare the training loss curves for the original and ablated versions of the model. Briefly describe the differences observed.





We see similar convergence in both the ablated and the standrad implimentations, although the standard loss converges at around 0.005 while the ablated train loss converges at 0.04. additionally the validation loss in the ablated case is higher at 0.03 unlike the 0.004 in the standard SRFlowGenerator. The loss diffrence is due to the cahnge in the loss function, adding in a weighted element of 0.2 * MGE which increases the overall loss.

3. Report quantitative performance metrics (e.g., accuracy, loss) for each version. Present the results in a clear format, such as a table.

Model	AVG. PSNR	AVG. SSIM	AVG. FID
Ablated SRFlowgenerator (+MGE loss)	25.0327 Std = 0.1650	0.7457 Std = 0.0093	29.9428 Std = 1.4676
SRFlowGenerator (our proposed)	24.9777 Std =0.0434	0.7430 Std = 0.0056	26.8959 Std = 1.0489

We see a small improovment in PSNR and SSIM metrics. Visually we see small improovment in the sharpness of the images. Although the FID score is lower for the ablated study.

We have a trade off with weighting in the MGE. The more its weight increases the stronger effect it has on the overall loss. The stronger the network will emphasize gradients and sharpness over image tactenss and structure.

4. Provide qualitative examples (e.g., prediction visualizations) to illustrate how the ablation affected performance. Was there an image where the ablated model performed better? Can you explain why?

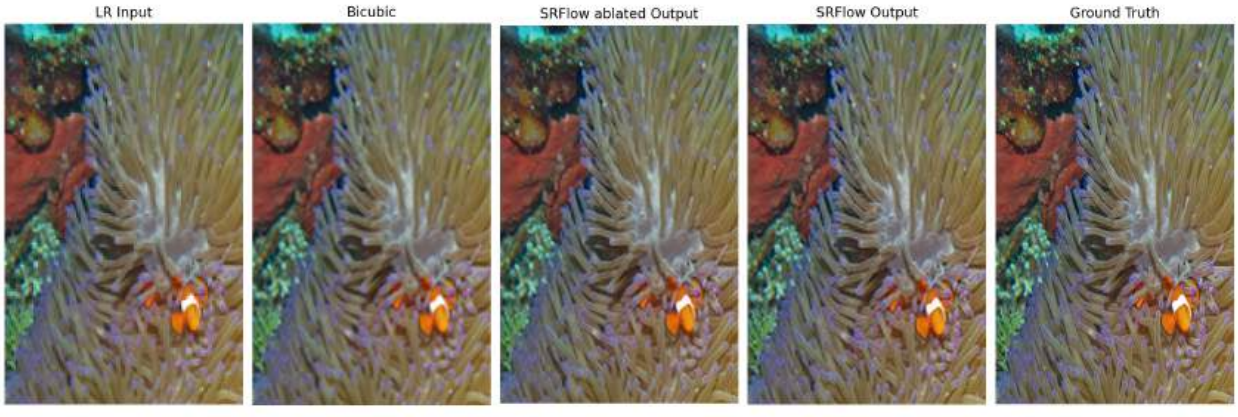
Illutstrating individual images were both models preformed best we see an about 0.2 dB increase in the “best SR images”. Improovment while incremental leads to conclude that adding a weighted MGE loss element improoved the results. Although A higher FID score means the distribution of the generated images is further away from the distribution of real images. This implies that while the MGE loss helps with objective sharpness and fidelity, it simultaneously introduces characteristics that make the generated images appear less realistic. The model might be over-sharpening edges or creating high-frequency textures that are not present in real images, leading to an artificial look.

Another intresting observeation is the that The gradient-focused loss might subtly distort overall color balance or fine textures in a way that deviates from natural image statistics.


LR Input
Bicubic
SRFlow ablated Output
SRFlow Output
Ground Truth


LR Input
Bicubic
SRFlow ablated Output
SRFlow Output
Ground Truth


LR Input
Bicubic
SRFlow ablated Output
SRFlow Output
Ground Truth


LR Input
Bicubic
SRFlow ablated Output
SRFlow Output
Ground Truth

Best #1 - LR

Best #1 - SR (PSNR=34.50)

Best #1 - Ground Truth

Best #2 - LR

Best #2 - SR (PSNR=34.07)

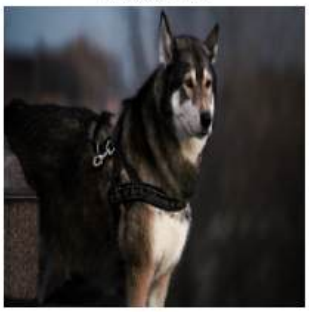
Best #2 - Ground Truth

Best #3 - LR

Best #3 - SR (PSNR=33.69)

Best #3 - Ground Truth

Best #4 - LR



Best #4 - SR (PSNR=33.64)

Best #4 - Ground Truth

Worst #1 - LR

Worst #1 - SR (PSNR=20.93)

Worst #1 - Ground Truth

Worst #2 - LR

Worst #2 - SR (PSNR=20.44)

Worst #2 - Ground Truth

Worst #3 - LR

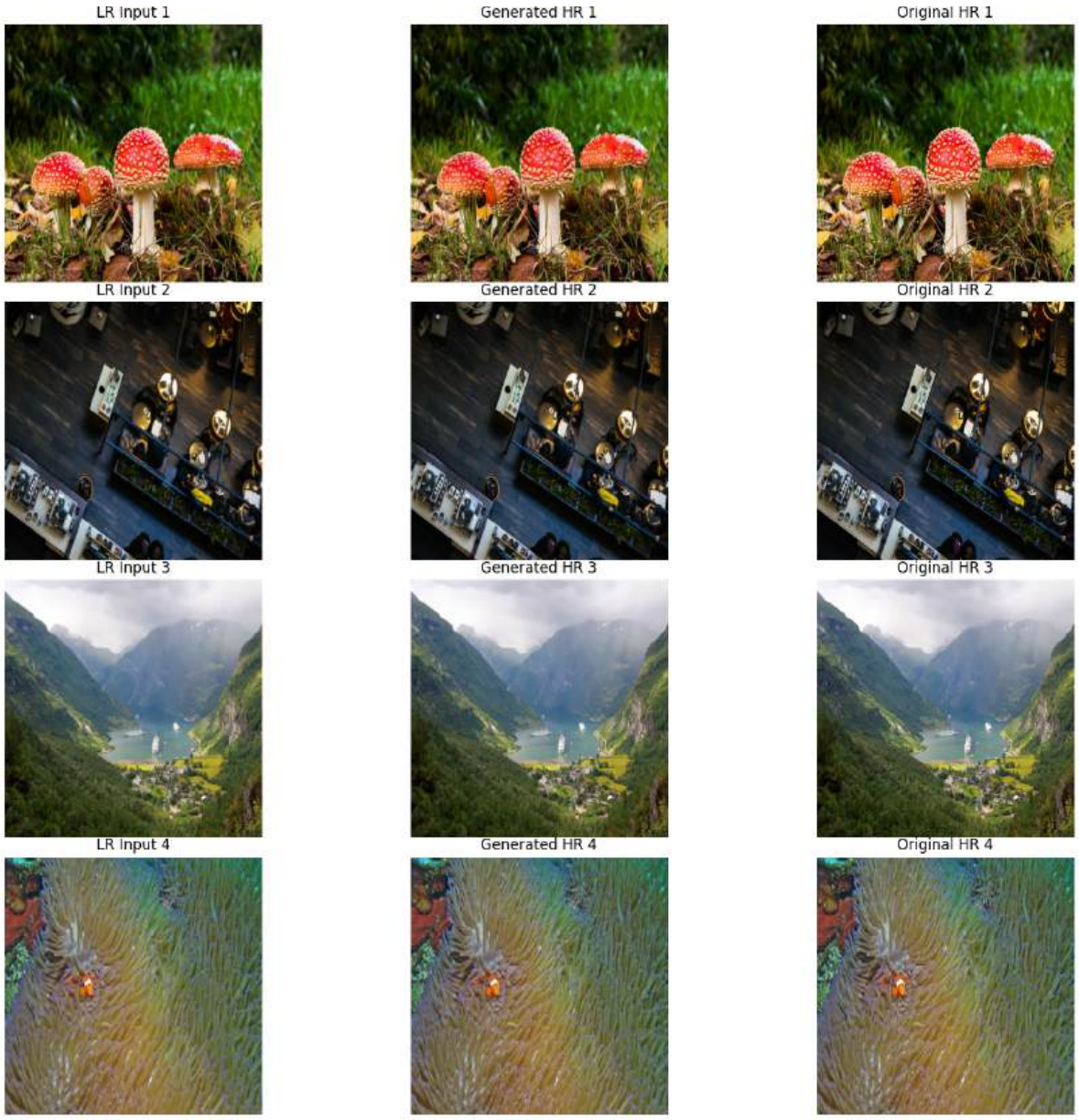
Worst #3 - SR (PSNR=20.18)

Worst #3 - Ground Truth

Worst #4 - LR

Worst #4 - SR (PSNR=19.17)

Worst #4 - Ground Truth


LR Input 1
Generated HR 1
Original HR 1
LR Input 2
Generated HR 2
Original HR 2
LR Input 3
Generated HR 3
Original HR 3
LR Input 4
Generated HR 4
Original HR 4

To validate this observation we retrain the model to see how SR with only MGE loss might preform. The weights now are $w_1 = 0, w_2 = 1$

$$Loss = 0 * MSE + 1 * MGE$$

MGE only SRFlow SR image

MSE only SRFlow SR image

Indeed prioritizing gradient sharpness we get images that look strikingly sharp but very off color. This discoloration results in incredibly low preformance metrics, which do emphasize the SR images coloration severly when calculating them.

Model	PSNR	SSIM	FID
SRFlowGenerator MGE loss only	12.9511 Std = 0.0923	0.2521 Std = 0.013	97.1411 Std = 1.2827

5. Summarize the insights gained. What does this experiment reveal about the role of the ablated component in your model's performance?

In total we see that changing the loss by weightinig the MGE loss component into the overall loss function results in imrpooved image sharpness. The trade off to this component is image coloration and color balance across its RGB channels. While MSE loss attempts to create the best SR image pixel by pixel MGE tries to achieve best possible sharpness and edges to represent the SR image as accurately as possible but at cost of disregardiong color importance across the channels.

Futher research will include fine tuning the weights w_1, w_2 to get the best possible results in terms of color and image sharpness. To find a balance in the trade off. Where each of the weights could be in range [-1,1].

Conclusions and Summary

In this project we attempted to propose an archtecture to solve the problem of super-resolution on the div2k dataset. We propose the SRFlowGenerator architecture which introduces a larger architecture with skip connections and RDB's to emphasize the depth of high frequency features in LR images. We see that indeed our propopsed method improoves over the SRModel vanilla implimentation drasticly – at the cost of sgnificantly increased training time but with much better FID score. The SRFlowGenerator preforms better in the proposed qualty metrics than the SRModel in all the test set iamges.

We then proceed to ablate the study and investigate the use of an weighted MGE loss to improove the sharpness of the SR generated images from the model. The results show that weghting the loss positivly in MGE icreases thw overall loss but also leads to incrimental improovments in the PSNR and SSIM of the generated images, we see that the images geenrated from this ablated training are sharper then the standard SRFlowGenerator trained only with MSE. We also see the effect of training purely with the MGE and conclude a trade off on the weighted loss. Where we either get very sharp – stark high gradient images, at the cost of discoloration. Further fine tuning is needed to find the perfect weights w_1, w_2 that result in best SR scores in terms of SSIM,FID and PSNR.

Refrences:

[1] div2k kaggle - https://www.kaggle.com/datasets/joe1995/div2k-dataset

[2] div2k – SRGAN (SRModel implimentation)
https://www.kaggle.com/code/piyutotakar/srgan-on-div2k

[3] SRFlow github - https://github.com/andreas128/SRFlow

[4] Ledig, Christian, et al. "Photo-realistic single image super-resolution using a generative adversarial network." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017.

[5] Lugmayr, Andreas, et al. "Srflow: Learning the super-resolution space with normalizing flow." Computer vision–ECCV 2020: 16th European conference, glasgow, UK, August 23–28, 2020, proceedings, part v 16. Springer International Publishing, 2020.